Lab 11 — Hybrid Cluster — Joining a Windows Worker Node

ID: K8S-LAB-11 · **Category:** Windows · **Level:** Enterprise · **Duration:** 90 min

Focus: Windows Kubernetes, kubelet.exe, HNS, containerd-windows, hybrid scheduling

Objective

Add a Windows Server 2022 node to a Linux control plane, schedule an IIS Pod on it, and debug HNS networking.

Concepts Covered

• Why Windows nodes need their own CNI (Calico-Windows or Flannel host-gw)

• OS-based nodeSelectors and tolerations

• HNS endpoints vs Linux veth pairs

Prerequisites

• Lab 1 cluster

• Windows Server 2022 VM with 4 vCPU / 8 GB RAM, joined to the same L2/L3 as the Linux node

Architecture

Same control plane, two node OS families. The scheduler uses kubernetes.io/os labels and nodeSelectors to keep Linux Pods on Linux nodes and Windows Pods on Windows nodes.

Lab Steps

01. Prepare the Windows node

Containers + Hyper-V are required for containerd-windows and HNS.

```
# On the Windows node (PowerShell as admin):
Install-WindowsFeature -Name Containers
Enable-WindowsOptionalFeature -Online -FeatureName Microsoft-Hyper-V -All -NoRestart
Restart-Computer
```

Expected: Server reboots, features enabled.

Internals: HNS (Host Network Service) is the Windows equivalent of cni0 + iptables. It's a Hyper-V virtual switch.

02. Install containerd, kubelet.exe, kubectl.exe

PrepareNode.ps1 installs the kubelet service, kubectl, and Calico/Flannel binaries.

```
# PowerShell:
```

```
curl.exe -LO https://github.com/containerd/containerd/releases/download/v1.7.13/containerd-1.7.13-windows-amd
tar -xvf containerd-1.7.13-windows-amd64.tar.gz -C $env:ProgramFiles\containerd
& $env:ProgramFiles\containerd\bin\containerd.exe --register-service
Start-Service containerd
# install kubelet & kubectl via SIG-Windows scripts:
curl.exe -LO https://raw.githubusercontent.com/kubernetes-sigs/sig-windows-tools/master/PrepareNode.ps1
.\PrepareNode.ps1 -KubernetesVersion v1.30.0
```

Expected: Service `kubelet` registered and Running.

Internals: kubelet.exe behaves like Linux kubelet but talks to containerd via the Windows named-pipe CRI socket.

03. Join the node

Same join command as Linux, just kubeadm.exe.

```
# Re-run `kubeadm token create --print-join-command` on the LINUX control plane,
# then on Windows in an elevated shell:
kubeadm.exe join <CP_IP>:6443 --token <t> --discovery-token-ca-cert-hash sha256:<h>
```

Expected: `This node has joined the cluster`

Internals: TLS bootstrap is identical; only the kubelet binary differs.

04. Verify and schedule an IIS pod

nodeSelector pins the Pod to Windows. The IIS image is ~5 GB so the first pull is slow.

```
kubectl get nodes -o wide
cat <<EOF | kubectl apply -f -
apiVersion: apps/v1
kind: Deployment
metadata: {name: iis}
spec:
  replicas: 1
  selector: {matchLabels: {app: iis}}
  template:
    metadata: {labels: {app: iis}}
    spec:
      nodeSelector: {kubernetes.io/os: windows}
      tolerations:
      - {key: "os", operator: "Equal", value: "windows", effect: "NoSchedule"}
      containers:
      - name: iis
        image: mcr.microsoft.com/windows/servercore/iis:windowsservercore-ltsc2022
        ports: [{containerPort: 80}]
EOF
kubectl get pods -o wide -l app=iis
```

Expected: Pod Running on the Windows node within ~10 min on first pull.

Internals: Windows containers are NOT lightweight — image layers are huge because each carries a chunk of the Windows base image.

Common Errors

Windows node NotReady — Cause: CNI (Calico/Flannel-Windows) not installed · Fix: Run sig-windows install-cni.ps1

Pod stuck ContainerCreating on Windows — Cause: HNS endpoint creation failed · Fix: Restart-Service hns; check `Get-HnsNetwork`

ImagePullBackOff for `mcr.microsoft.com/windows/...` — Cause: Pulled to a Linux node by accident · Fix: Add nodeSelector kubernetes.io/os=windows

Troubleshooting

• `Get-HnsNetwork`, `Get-HnsEndpoint`

• Windows event log: kubelet writes to Application

• `crictl --runtime-endpoint npipe:////./pipe/containerd-containerd ps`

Enterprise Relevance

Mixed Linux/Windows clusters are common in enterprises with .NET Framework legacy. Modernization rarely deletes Windows workloads — it just orchestrates them.

Validation

• A Windows Pod runs alongside Linux Pods in the same cluster

• You can reach the IIS Pod's PodIP from a Linux Pod

Related Incidents & Guide Sections

Incidents: K8S-INC-3002

• Linux ↔ Windows hybrid

• Join Windows node

Questions

Beginner

Q. What label distinguishes a Windows node?

A. kubernetes.io/os=windows

Intermediate

Q. Why is the IIS image multi-GB?

A. It bundles the Windows Server Core base image — there's no shared host kernel.

Advanced

Q. What does HNS do?

A. Provides the virtual switch and endpoints that map to Pod network namespaces on Windows.

Scenario

Q. All Windows pods go NotReady at once. First check?

A. Get-Service hns on each Windows node, then the Windows CNI pod.

Viva

Q. Can a Pod contain both a Linux and a Windows container?

A. No — a Pod runs on a single node, which has a single OS.